Reinforcement Learning for Snake Game

Nadav Izhaki - 329087852

Lior Yanwo - 207918103

Department of Computer Science, Holon Institute of Technology (HIT).

Course: Deep Learning.

GitHub Link: https://github.com/nadav0912/RF-Snake-game.git

27.06.2026

Introduction and Project Background

The core objective of this project was to take a basic, functional Reinforcement Learning model and engineer it into an advanced agent capable of high-level performance.

I started by implementing the baseline model, a simple Deep Q-Network (DQN) for the game of Snake, based on an online tutorial [Link].This initial version used an 11-parameter linear model that successfully learned the basic mechanics but quickly hit a strict performance ceiling, struggling to plan ahead or avoid its own growing body.

To achieve significantly better results, I completely overhauled the agent's design. Beyond upgrading the network to a custom Convolutional Neural Network (CNN) hybrid, I optimized the state representation, refined the reward system, and improved the training logic. Through this comprehensive architectural upgrade, the model gained the strategic depth and spatial awareness needed to navigate complex scenarios and achieve much higher scores.

The Baseline Model (V1)

The initial agent relied on a highly simplified state representation. The state was an 11-element boolean array that only captured the snake's immediate surroundings:`[danger straight, danger right, danger left, direction left, direction right, direction up, direction down, food left, food right, food up, food down]`.

This raw data was fed into a standard feed-forward Deep Q-Network consisting of an input layer, a single hidden layer of 256 neurons with a ReLU activation function, and an output layer of 3 neurons representing the possible actions (straight, right, or left). The reward system was strictly reactive, granting +10 points for eating food, -10 points for a game-over event (collision or timeout), and 0 points for any standard movement step.

The Inherent Limitation: While this setup allowed the agent to quickly learn basic survival and food-seeking behavior, it suffered from a fundamental flaw of short-sightedness. Because the model could only perceive its immediate adjacent blocks, it lacked the global spatial awareness needed to understand the board's layout. Consequently, it struggled simply to navigate around its own tail or plan a safe route to the food without immediately trapping itself, strictly capping its maximum performance.

The Final Architecture (V2)

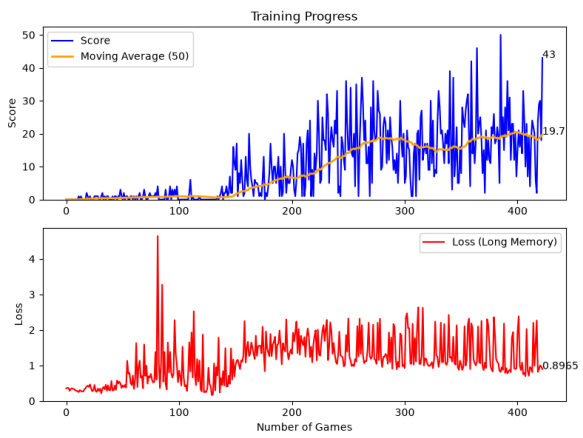
To shatter the performance ceiling of the baseline model, the agent required a fundamental shift in how it perceived its environment. Rather than relying on a narrow, localized view, the new architecture introduces a dual-input hybrid state. The crown jewel of this upgrade is the visual state, a 3D matrix composed of three distinct channels. The first channel isolates the snake's head, and the second maps all physical obstacles (the body and walls), both utilizing binary '1's and '0's to provide precise spatial mapping. The third channel acts as a continuous navigation gradient, providing a spatial heatmap based on the Manhattan distance to the food. However, vision alone can be computationally abstract for immediate reactions. To ground the model, a logical state consisting of 12 boolean parameters (similar to the state in the first version) is injected in parallel. This serves as a hard-coded reflex system, providing explicit data on current movement direction, absolute dangers, and relative target location.

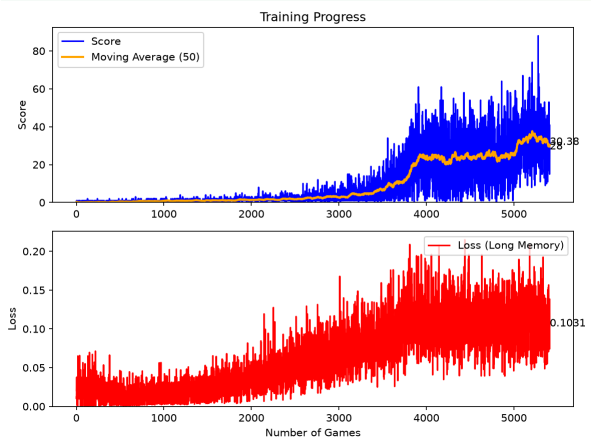
Designing the network to process this rich data required careful architectural scaling. Early experimental iterations proved that the game's spatial complexity demanded a significantly larger and deeper model to truly understand the board. The visual tensor is fed through a Convolutional Neural Network (CNN), strategically utilizing MaxPool and Dropout layers to extract complex geometric patterns like the shape of the snake's growing tail while actively preventing the network from merely memorizing specific board states. As the visual data is compressed into a dense feature vector, the architecture employs a "Late Fusion" technique. This compressed visual funnel is concatenated with the 12 logical

parameters just before the final fully connected layers. This fusion allows the agent to synchronize long-term visual pathfinding with immediate logical survival instincts. Crucially, the output action space was also upgraded from three relative movements to four absolute directions (Up, Down, Left, Right), granting the agent unambiguous navigational control.

The core of the agent's learning process is the Deep Q-Learning algorithm, optimized by the Adam optimizer and a Mean Squared Error (MSE) loss function. To ensure training stability and prevent the agent from chasing a constantly moving target, a dual-network approach was implemented. A secondary, "frozen" Target Network provides consistent learning goals, syncing its weights with the actively learning primary model only once every 1,000 steps. A critical component of this learning loop is the Epsilon-greedy strategy, which dictates the agent's randomness. Initially, the agent operates with a high degree of randomness, exploring the board chaotically to build a diverse memory buffer of varied scenarios. As training progresses, this randomness gracefully decays, allowing the increasingly confident neural network to take control and exploit its learned strategies. To shape this behavior, the reward function was initially calibrated with both primary and distance-based micro-rewards: granting +10 points for securing food, a harsh -10 point penalty for fatal collisions, +0.1 for moving closer to the food, and -0.2 for moving away. To further refine the model's strategy, a secondary training phase (Curriculum Learning) was initiated from a late training checkpoint. During this phase, the distance-based micro-rewards were completely removed (leaving only the absolute +10 and -10 rewards) and the Learning Rate (LR) was lowered. This adjustment forced the model to rely on its spatial awareness rather than immediate gratification, successfully pushing the moving average up to ~35 points without requiring additional architectural changes. However, while this secondary training significantly improved overall performance, it did not completely eliminate the agent's tendency to occasionally enter geometric loops as it grew longer, highlighting a persistent challenge in navigating complex self-created obstacles.

Results and Comparative Analysis





The training data highlights a clear trade-off between model complexity and performance. The lightweight V1 baseline learns quickly but plateaus after a few hundred games (moving average: 19.7, peak score: 50). In contrast, the deep V2 Hybrid model requires significantly more training time (5,500 games) to process its visual state space, but achieves a vastly superior performance ceiling (moving average: ~33.3, peak score: >80) alongside a stable loss convergence (0.0272).

Note: We continued training the model V2 from the same point, but changed the reward for regular actions to 0 and lowered the Learning Rate (LR). This caused further improvement in the agent, bringing the results to a moving average of ~35 and a peak score of nearly 90.

Conclusion

Upgrading to a dual-input CNN hybrid, alongside comprehensive improvements to the reward system and training logic, significantly enhanced the agent's capabilities. Compared to the V1 baseline, the new AI is highly capable of long-term pathfinding and successfully avoids its own growing body. However, despite these major advancements, the agent cannot completely beat the game yet, the immense geometric complexity of late-game scenarios remains a profound challenge.